

BACHARELADO EM ENGENHARIA DE SOFTWARE

Disciplina: Design de Software

Especificação do Projeto – Avaliação Somativa (RA1)

CoCar – Plataforma de Caronas Corporativas

Equipe: Projeto RA1 5

Turma B

Integrantes:

João Pedro Gadens Mosson

João Pedro Magri Pozzan

Eduardo Skoroboatei Gomes

Victor Oliveira Medeiros

Renan Cassou Rodrigues

Professora: Andreza de Sousa Vieira

Data de entrega: 31/08/2026

Curitiba

2. Contextualização do Projeto

2.1 Domínio da Aplicação

O CoCar é uma plataforma web de mobilidade corporativa que conecta motoristas e passageiros de uma mesma organização para a realização de caronas compartilhadas. O sistema permite que colaboradores cadastrem seus trajetos recorrentes (rotas de casa para o trabalho, por exemplo), sejam automaticamente cruzados com pessoas que fazem percursos semelhantes, e dividam os custos da viagem de forma digital, através de uma carteira virtual integrada.

2.2 Problema

O deslocamento diário de colaboradores para o ambiente de trabalho é, em muitos casos, ineficiente e custoso: veículos particulares circulam com apenas um ocupante, gerando gastos individuais elevados com combustível e pedágio, além de contribuir para o trânsito e a poluição nos grandes centros urbanos. Ao mesmo tempo, iniciativas informais de carona entre colegas esbarram na falta de uma ferramenta que garanta segurança na identificação dos participantes (por vínculo comprovado com a mesma empresa), facilite o encontro entre quem oferece e quem precisa de carona, e automatize a divisão justa dos custos o que faz com que esse potencial de economia e sustentabilidade acabe não sendo aproveitado.

2.3 Objetivos do Sistema

O sistema tem como objetivo geral conectar motoristas e passageiros de uma mesma organização que realizam trajetos semelhantes, otimizando o uso de veículos particulares. De forma específica, o CoCar busca:

- Reduzir os custos de deslocamento diário de motoristas e passageiros por meio do rateio automático de despesas como combustível e pedágio;
- Promover a mobilidade urbana sustentável, diminuindo a quantidade de veículos circulando com baixa ocupação e, consequentemente, a emissão de poluentes;
- Oferecer às organizações parceiras um ambiente corporativo controlado, com validação de motoristas e visibilidade sobre o engajamento dos colaboradores no programa de caronas, incluindo métricas de impacto ambiental (redução estimada de CO2) e mecanismos de incentivo (benefícios) para quem oferece caronas com frequência.

3. Lista de Requisitos Funcionais

A tabela abaixo apresenta a relação completa e numerada dos requisitos funcionais do sistema.

ID	Descrição do Requisito Funcional
RF01	O sistema deve permitir que o motorista realize seu cadastro informando dados pessoais, CNH e veículo, ficando pendente de aprovação até validação do administrador.
RF02	O sistema deve permitir que o usuário (carona/passageiro) realize seu cadastro informando nome, e-mail corporativo, CPF e senha.
RF03	O sistema deve permitir que os usuários realizem login e sejam autenticados de acordo com seu tipo de perfil (motorista, passageiro ou administrador).
RF04	O sistema deve permitir a recuperação e redefinição de senha em caso de esquecimento.
RF05	O sistema deve permitir que um administrador realize o cadastro da empresa/organização, informando CNPJ e domínio de e-mail corporativo válidos.
RF06	O sistema deve permitir que o administrador da empresa valide e aprove o cadastro de motoristas, liberando seu acesso completo às funcionalidades.
RF07	O sistema deve permitir que o administrador visualize a lista de motoristas cadastrados, com status (ativo/inativo) e filtro de busca por nome.
RF08	O sistema deve permitir que o usuário insira saldo em sua carteira virtual através de pagamento digital.
RF09	O sistema deve permitir que o motorista cadastre (poste) um trajeto informando origem e destino, exibindo a rota no mapa.
RF10	O sistema deve permitir que o motorista crie grupos de carona fixa, definindo dias da semana, horários e número de vagas.
RF11	O sistema deve permitir que o passageiro se candidate a uma carona informando ponto de partida e destino.
RF12	O sistema deve permitir que o motorista visualize e selecione os passageiros candidatos para vincular à sua carona.
RF13	O sistema deve permitir que o passageiro visualize a confirmação e reserva da carona assim que um motorista aceitar seu pedido.
RF14	O sistema deve permitir que o passageiro visualize em tempo real o trajeto do motorista e a quantidade de paradas restantes até seu embarque.
RF15	O sistema deve registrar automaticamente as transações financeiras da carteira virtual (cobranças, estornos) referentes a cada carona realizada.
RF16	O sistema deve permitir que o usuário consulte o histórico de suas viagens e transações realizadas.
RF17	O sistema deve permitir que o administrador da organização visualize métricas de redução de poluentes (estimativa de CO2 e km compartilhados) em um dashboard.
RF18	O sistema deve permitir que o administrador da organização cadastre benefícios para motoristas, associados a uma meta de quilometragem.
RF19	O sistema deve conceder automaticamente o benefício cadastrado ao motorista que atingir a meta de quilometragem definida.
RF20	O sistema deve validar o formato de documentos (CPF, CNH e CNPJ) informados nos formulários de cadastro, através do cálculo dos dígitos verificadores.

4. Diagrama de Componentes e Descrição

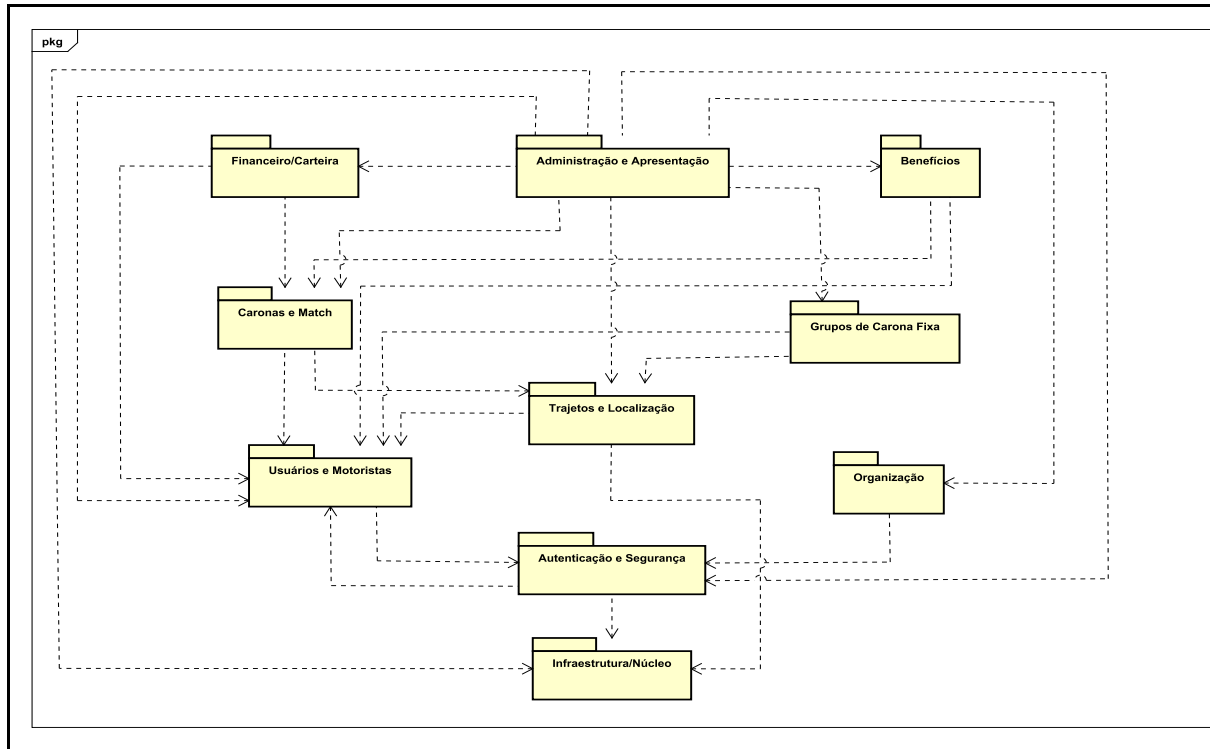
4.1 Organização em Pacotes

O diagrama de componentes do CoCar foi organizado em 10 pacotes, definidos a partir do critério de agrupamento por responsabilidade de negócio: cada pacote reúne os componentes sejam eles classes de aplicação, tabelas de persistência, bibliotecas ou enumerações que atuam sobre um mesmo contexto funcional do sistema, independentemente da camada técnica (Controller, Service, Model, etc.) a que pertenciam na implementação original. Essa abordagem garante coesão dentro de cada pacote e reduz o acoplamento entre pacotes distintos, que se comunicam exclusivamente através de interfaces bem definidas.

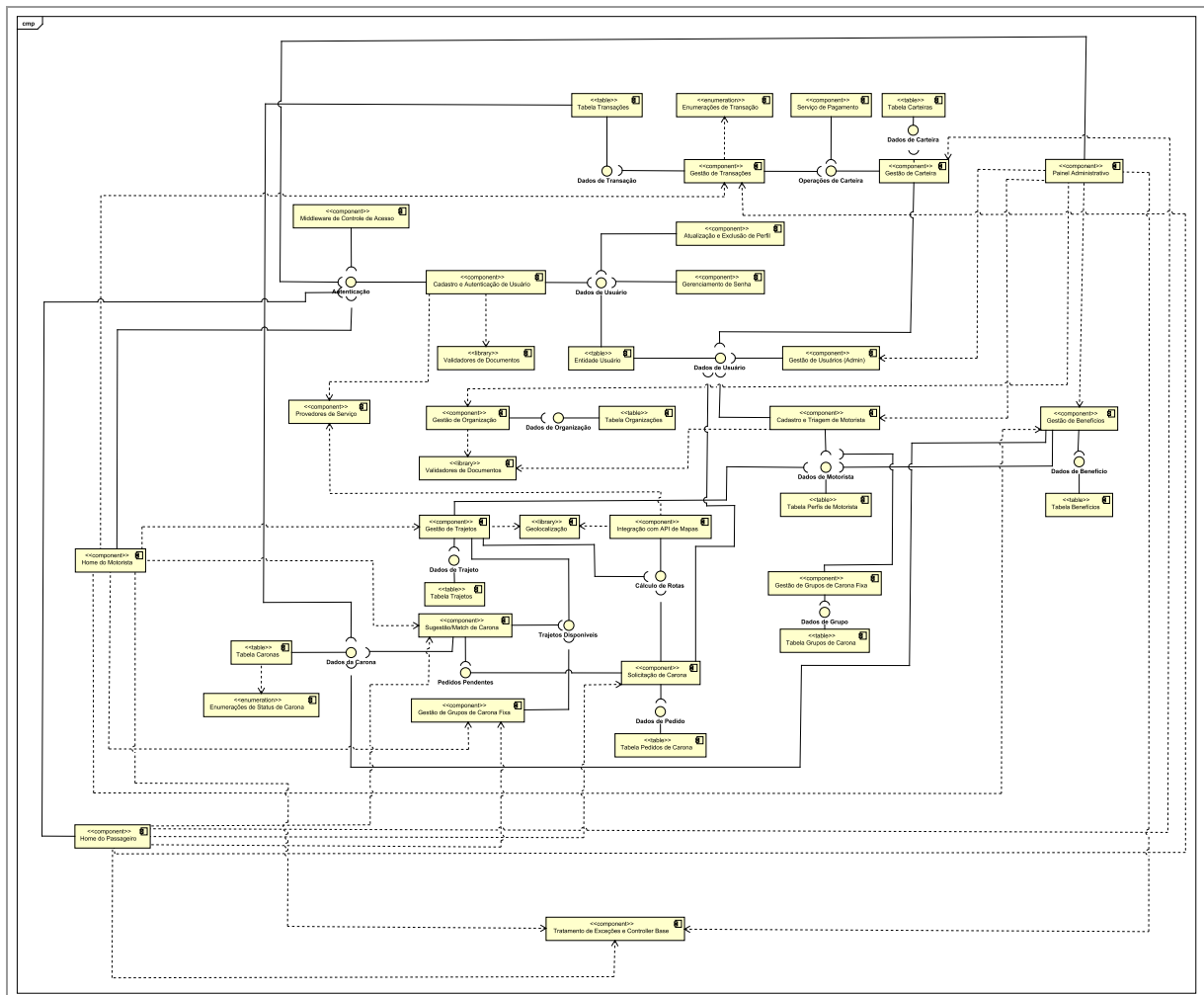
Os pacotes definidos são:

- **Autenticação e Segurança:** componentes responsáveis pelo cadastro, login, gerenciamento de senha e validação de documentos (CPF, CNPJ, CNH) dos usuários da plataforma.
- **Organização:** componentes responsáveis pelo cadastro e gestão das empresas parceiras que utilizam o sistema.
- **Usuários e Motoristas:** componentes responsáveis pelo cadastro, triagem e administração de usuários e do perfil específico de motorista.
- **Trajeto e Localização:** componentes responsáveis pelo cadastro de rotas, geolocalização e integração com a API externa de mapas.
- **Caronas e Match:** componentes responsáveis pela solicitação de carona, pelo cruzamento (match) entre motoristas e passageiros e pelo ciclo de vida da carona.
- **Grupos de Carona Fixa:** componentes responsáveis pela criação e gestão de grupos de caronas recorrentes.
- **Financeiro/Carteira:** componentes responsáveis pela carteira virtual, processamento de pagamentos e registro de transações.
- **Benefícios:** componentes responsáveis pelo cadastro de benefícios e sua concessão automática a motoristas que atingem metas.
- **Administração e Apresentação:** componentes responsáveis pelos painéis e telas iniciais voltados a cada perfil de usuário (administrador, motorista e passageiro).
- **Infraestrutura/Núcleo:** componentes transversais de suporte à aplicação, como provedores de serviço e tratamento de exceções, utilizados pelos demais pacotes.

4.2 Diagrama de Pacotes

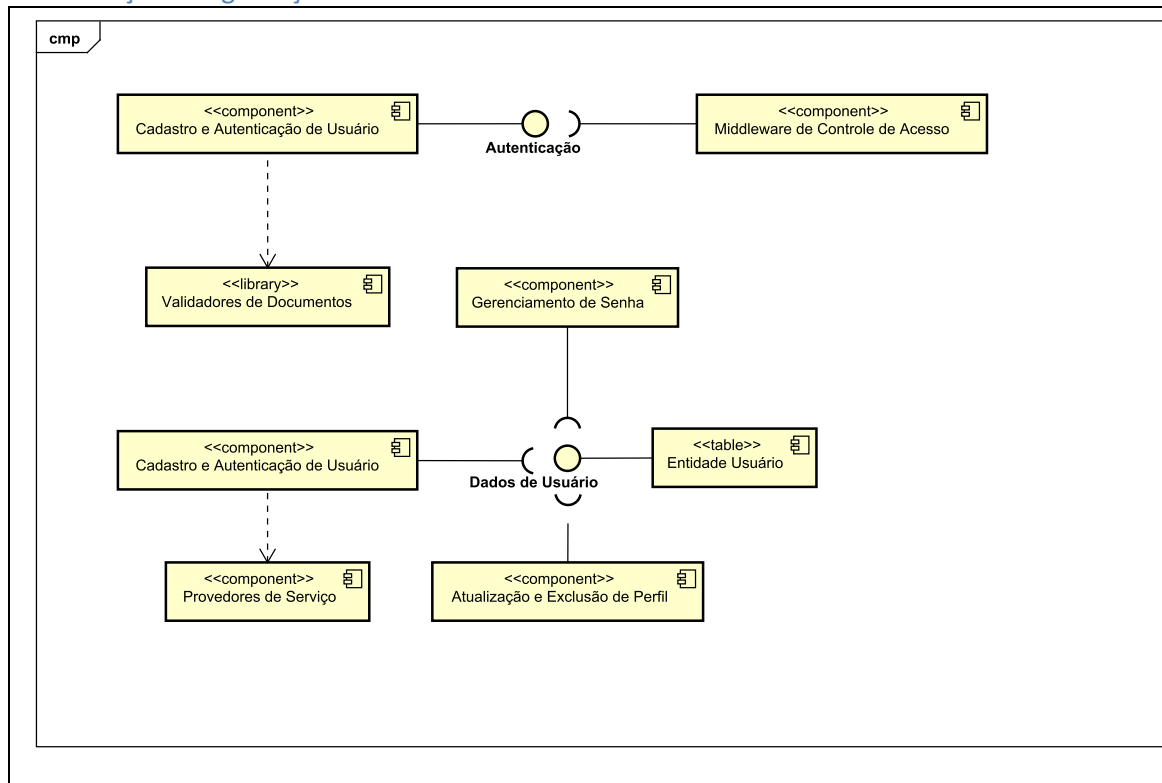


4.3 Diagrama de Componentes Completo

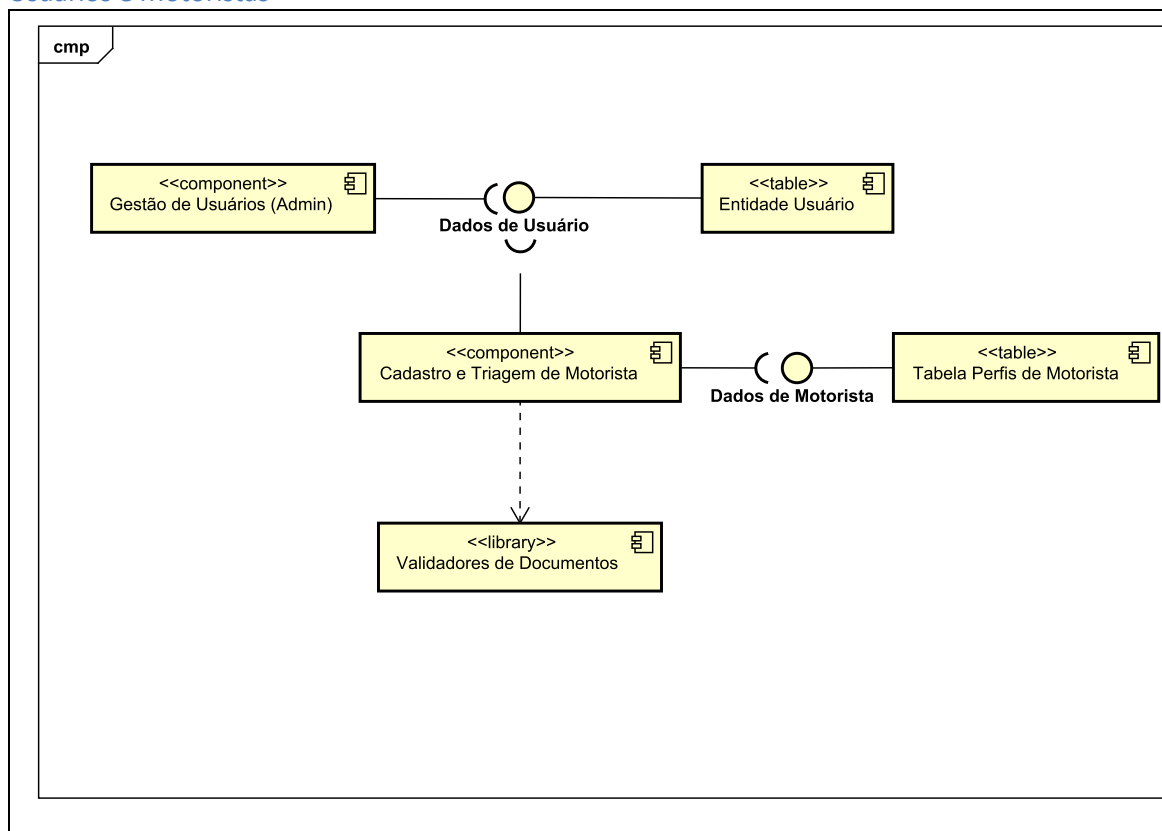


4.4 Diagramas por Pacote

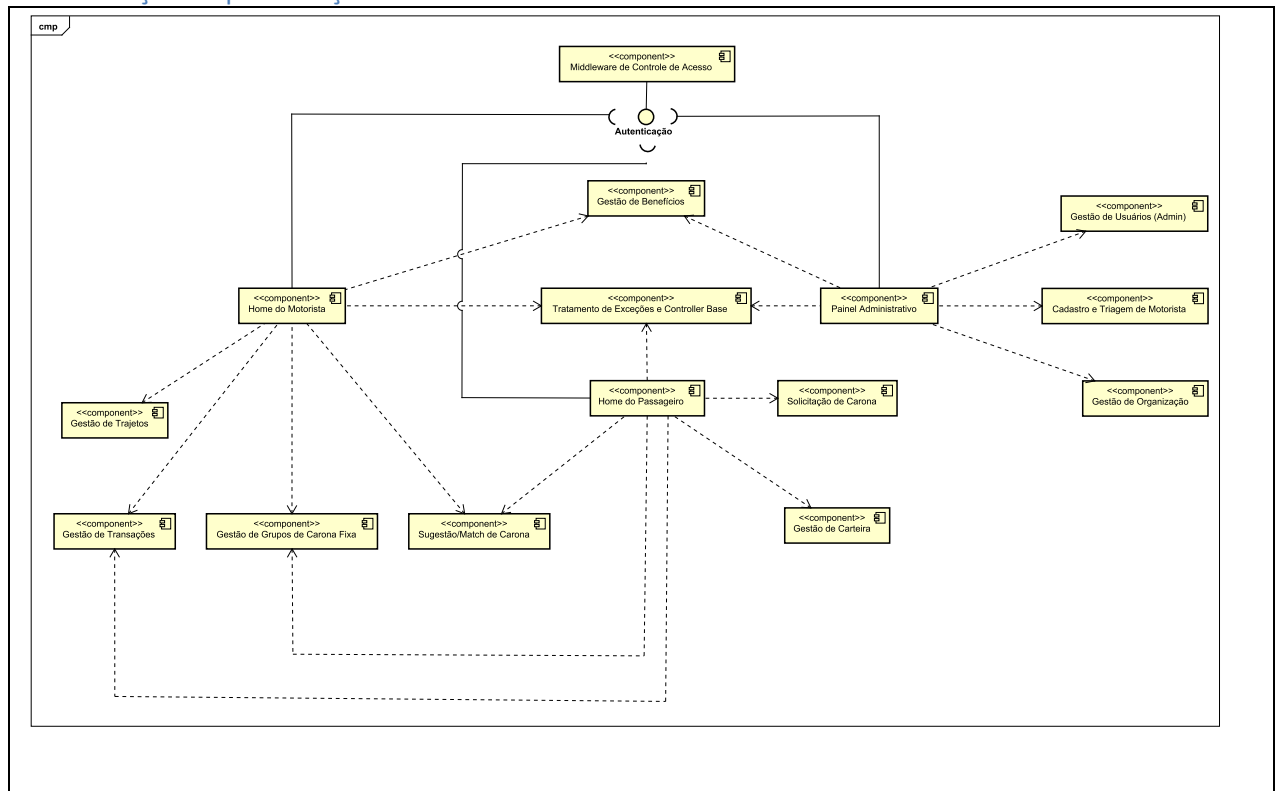
1. Autenticação e Segurança



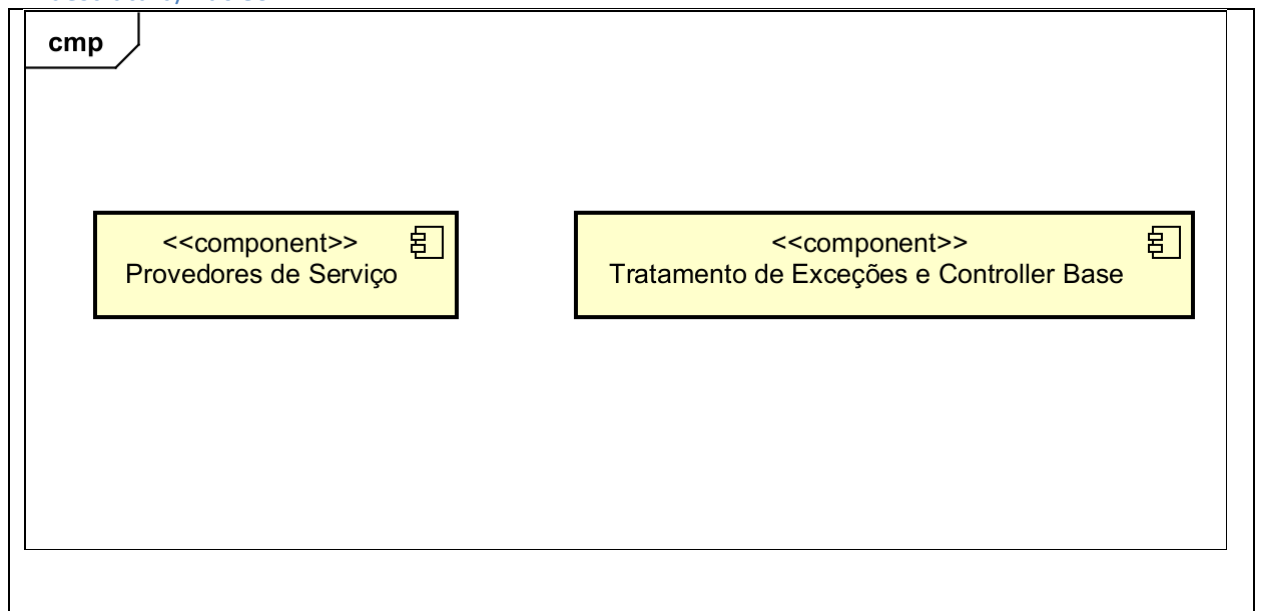
2. Usuários e Motoristas



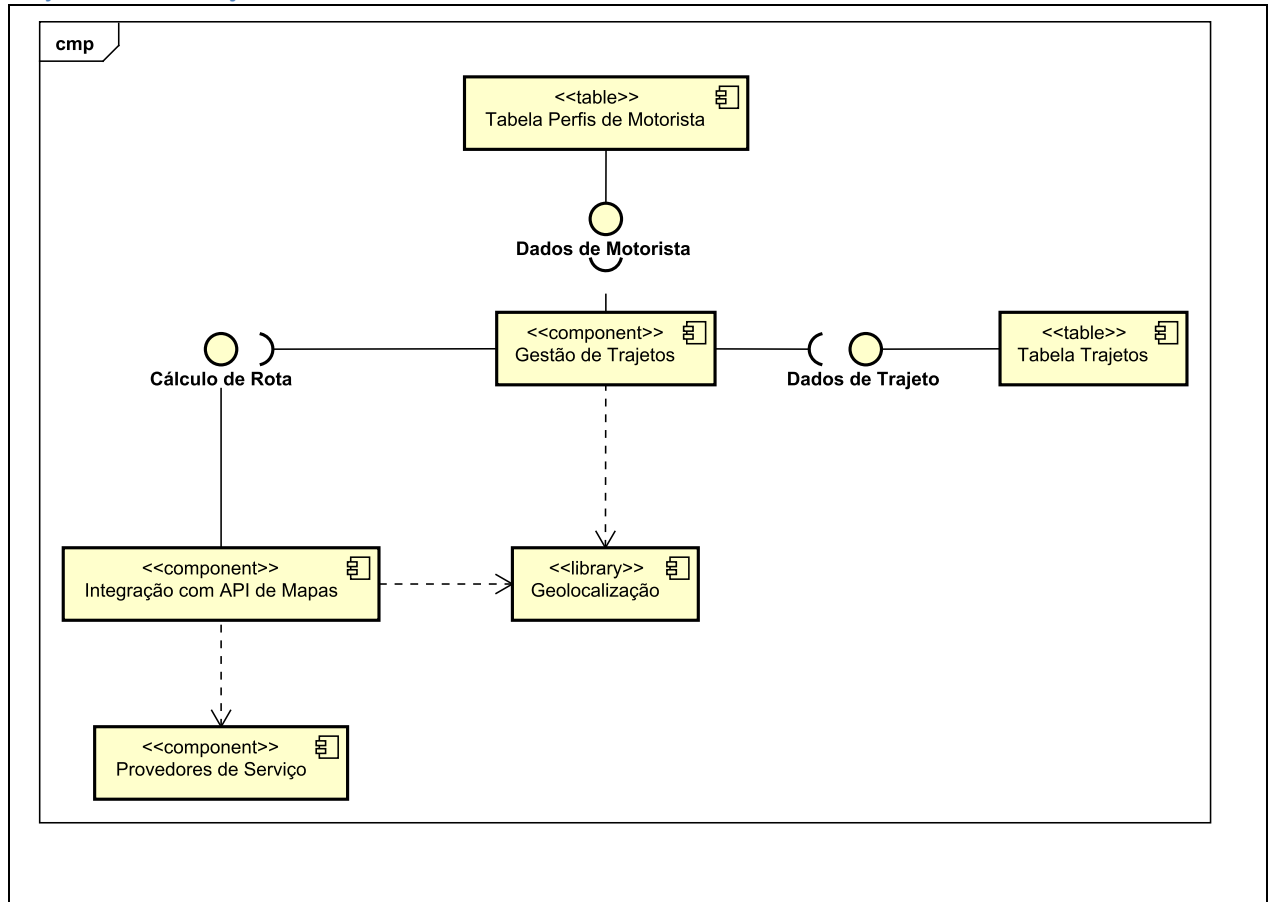
3. Administração e Apresentação



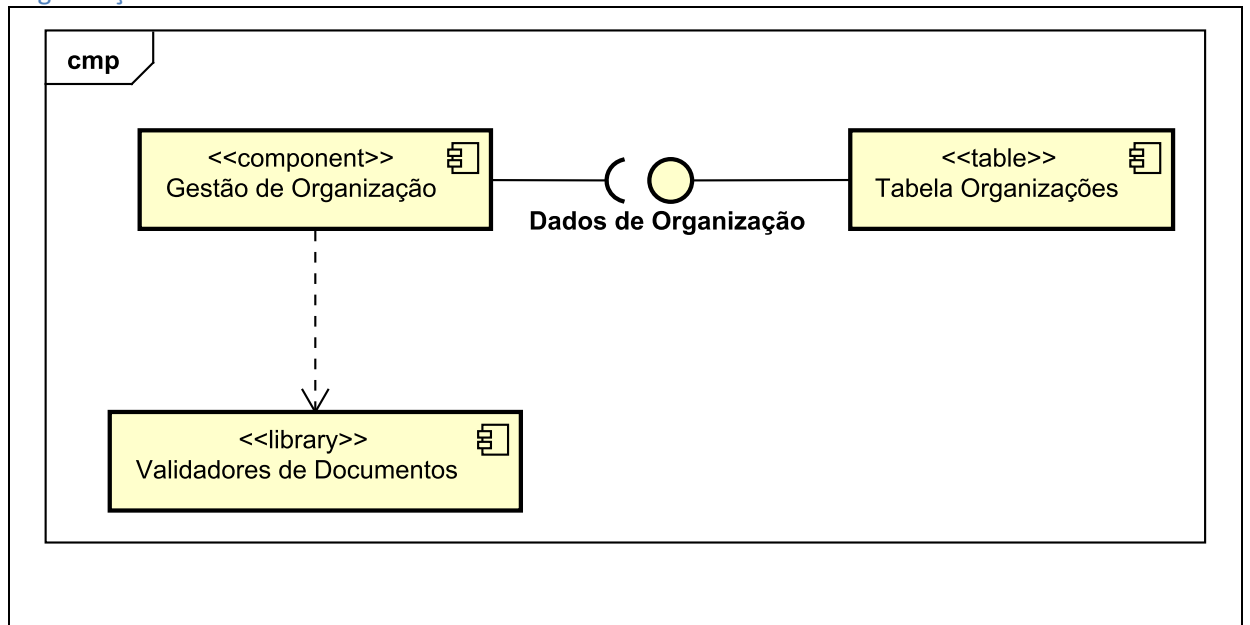
4. Infraestrutura/Núcleo



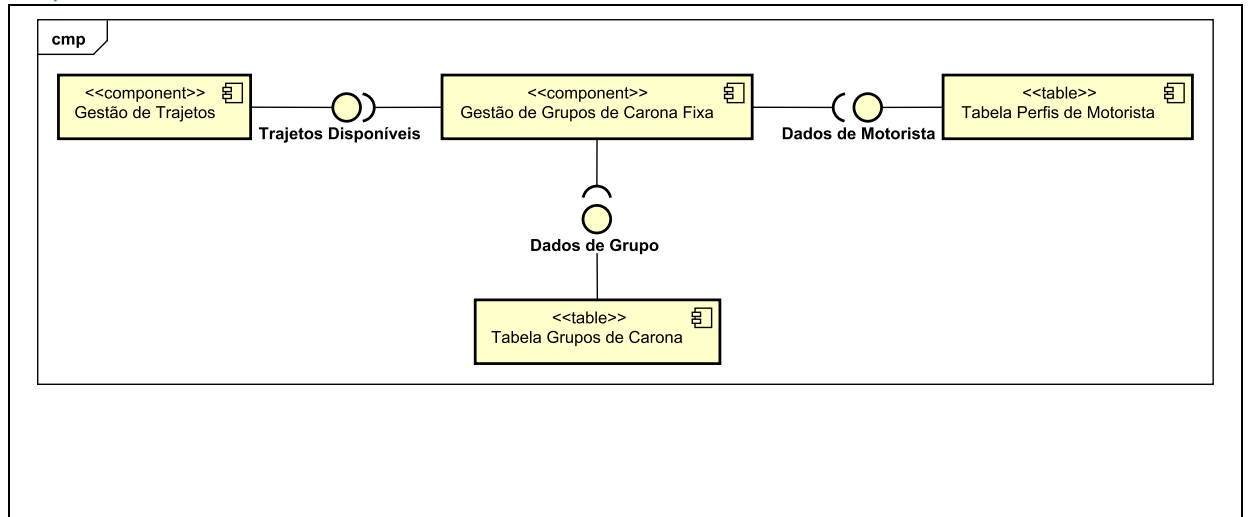
5. Trajetos e Localização



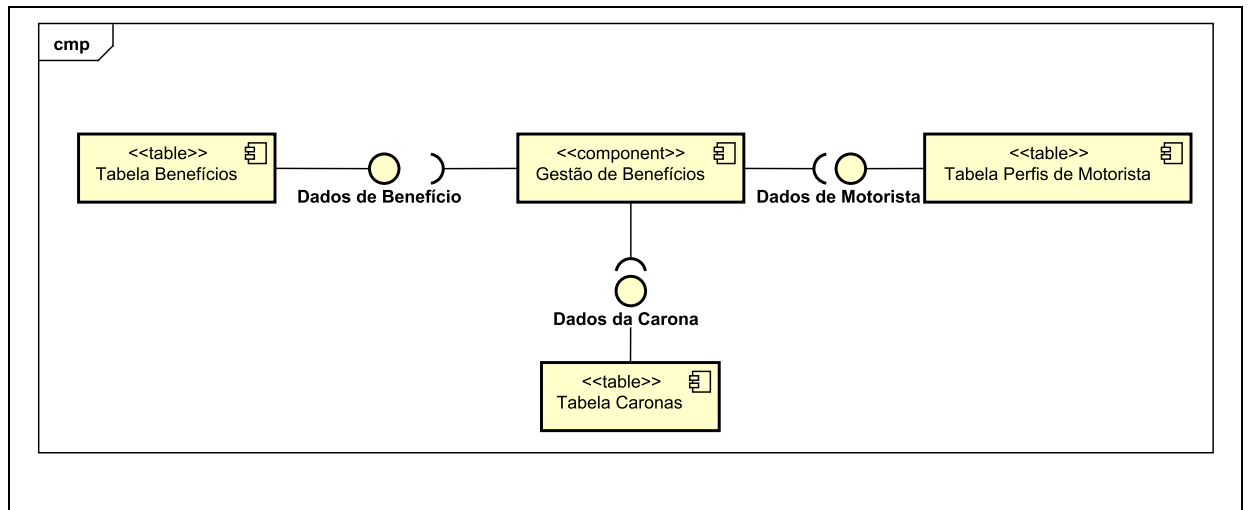
6. Organização



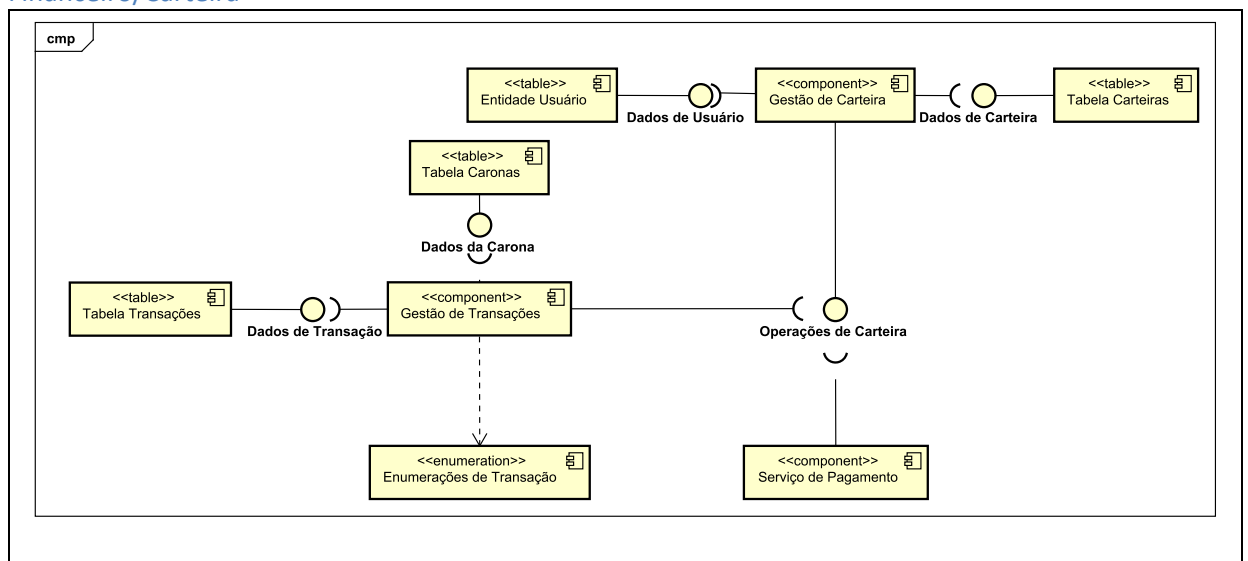
7. Grupos de Carona Fixa



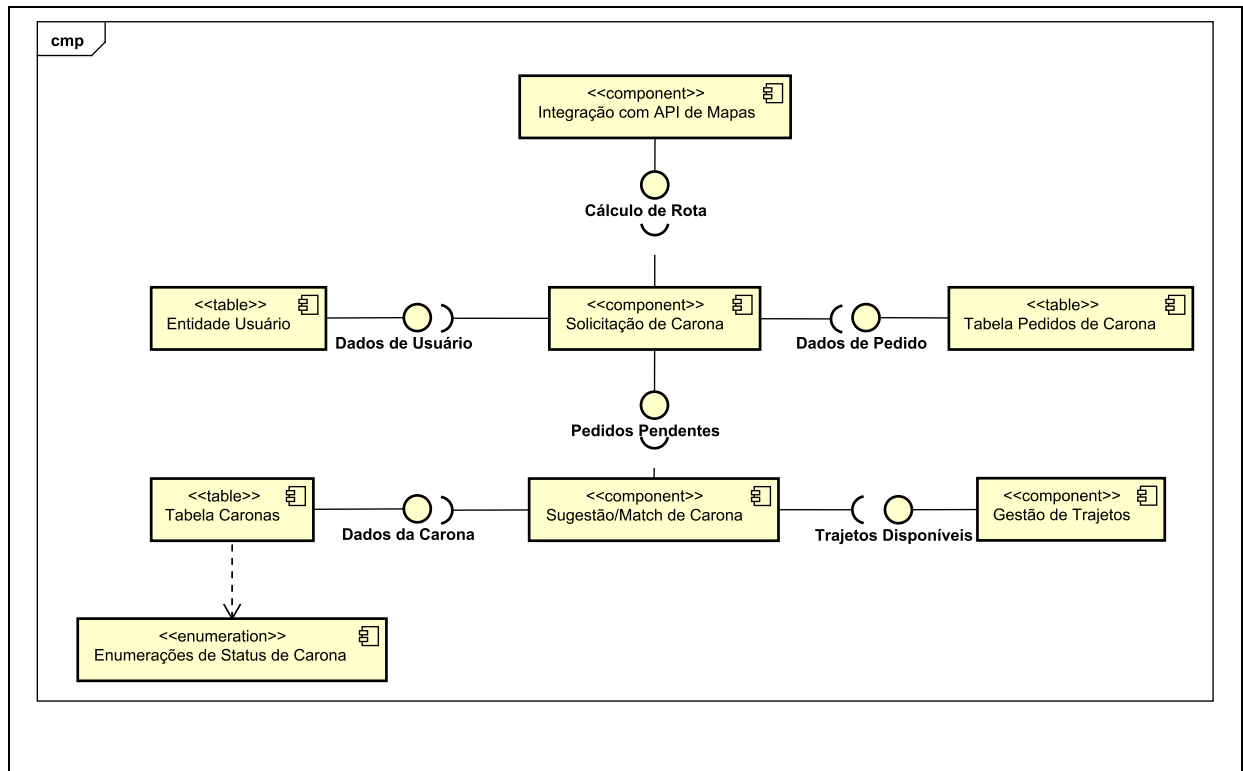
8. Benefícios



9. Financeiro/Carteira



10. Caronas e Match



4.5 Descrição Técnica

O Diagrama de Componentes do CoCar foi desenvolvido a partir da arquitetura real do sistema, implementado em Laravel, mapeando as 62 classes do código-fonte e as 11 tabelas de domínio em 35 componentes, distribuídos em 10 pacotes organizados por responsabilidade de negócio (conforme critério detalhado na Seção 4.1). No total, o diagrama contempla 58 conexões, sendo 24 delas representadas como interfaces (quando um serviço é reaproveitado por mais de um componente consumidor) e as demais como dependências simples (quando o uso é específico de um único componente), seguindo a convenção de notação UML2.5 estudada na disciplina.

Visão arquitetural geral

Analisando as dependências agregadas por pacote, o sistema revela uma estrutura em camadas lógicas, ainda que não estritamente hierárquica: o pacote Infraestrutura/Núcleo funciona como base do sistema, não dependendo de nenhum outro pacote, e fornecendo suporte transversal (provedores de serviço e tratamento de exceções) para os demais. Acima dele está o pacote Autenticação e Segurança, consumido por praticamente todos os pacotes de domínio. Os pacotes de domínio de negócio (Usuários e Motoristas, Organização, Trajetos e Localização, Caronas e Match, Grupos de Carona Fixa, Financeiro/Carteira e Benefícios) se apoiam uns nos outros de forma mais horizontal, refletindo as regras de negócio do sistema por exemplo, uma carona só pode ser cobrada (Financeiro) depois de confirmada (Caronas e Match), e um benefício só é concedido (Benefícios) a

partir do histórico de caronas realizadas. No topo da arquitetura está o pacote Administração e Apresentação, que concentra as telas do sistema (painéis e homes) e consome funcionalidades de praticamente todos os demais pacotes, papel condizente com sua natureza de camada de apresentação.

Descrição por pacote

- **Autenticação e Segurança:** reúne os componentes responsáveis pelo ciclo de vida da conta do usuário cadastro, login, gerenciamento de senha, atualização/exclusão de perfil e controle de acesso via middleware. O componente Cadastro e Autenticação de Usuário fornece a interface Dados de Usuário, consumida pelos demais componentes do pacote e por vários componentes de outros pacotes (Usuários e Motoristas, Caronas e Match, Financeiro/Carteira) que precisam acessar dados do usuário autenticado. O Middleware de Controle de Acesso fornece a interface Autenticação, reaproveitada pelas três telas do pacote de Apresentação.
- **Usuários e Motoristas:** centraliza a tabela Entidade Usuário e o cadastro/triagem de motoristas. O componente Cadastro e Triagem de Motorista fornece a interface Dados de Motorista, uma das mais reutilizadas do sistema consumida por Trajetos, Grupos de Carona Fixa e Benefícios, já que a validação de vínculo com um motorista aprovado é pré-requisito para essas funcionalidades.
- **Organização:** isolado propositalmente da camada de Usuários, contém o cadastro da empresa parceira, com dependência de Validadores de Documentos (para validação de CNPJ) e é consumido apenas pelo Painel Administrativo.
- **Trajetos e Localização:** reúne a gestão de rotas e a integração com serviço externo de mapas. A interface Cálculo de Rota, fornecida pela Integração com API de Mapas, é reaproveitada tanto pela Gestão de Trajetos quanto pela Solicitação de Carona, evitando duplicidade de lógica de geocodificação.
- **Caronas e Match:** é o núcleo funcional do sistema, contendo o componente Sugestão/Match de Carona, que requer três interfaces diferentes (Trajetos Disponíveis, Pedidos Pendentes e Dados da Carona) para realizar o cruzamento entre motoristas e passageiros refletindo a complexidade de negócio dessa funcionalidade central.
- **Grupos de Carona Fixa:** funcionalidade correlata a Caronas e Match, porém tratada como pacote separado por representar um fluxo de negócio distinto (caronas recorrentes vs. avulsas). Depende de Usuários e Motoristas e reaproveita a interface Trajetos Disponíveis de Trajetos e Localização.
- **Financeiro/Carteira:** concentra carteira virtual, pagamento e transações. Internamente, o Serviço de Pagamento fornece a interface Operações de Carteira, consumida também pela Gestão de Transações assim, qualquer alteração de saldo do sistema passa por um único ponto de acesso controlado.
- **Benefícios:** pacote mais simples, dependente de Usuários e Motoristas (para identificar o motorista elegível) e de Caronas e Match (para acessar o histórico de corridas usado no cálculo da meta).
- **Administração e Apresentação:** concentra as três telas do sistema. Diferentemente dos demais pacotes, a maioria das conexões aqui é representada por dependência simples (e não interface), pois cada tela consome funcionalidades de forma pontual, sem que esse consumo seja, por si, um contrato reutilizável por terceiros exceto pelo acesso ao Middleware de Autenticação, comum às três telas.
- **Infraestrutura/Núcleo:** base transversal do sistema, sem dependências próprias, fornecendo suporte a componentes que interagem com serviços externos ou que expõem rotas ao usuário.

Padrão de interfaces vs. dependências

A decisão de quando usar interface (círculo/meia-lua) ou dependência simples (seta tracejada) seguiu um critério objetivo: interfaces foram adotadas sempre que um mesmo recurso é fornecido a múltiplos consumidores como o acesso às tabelas de domínio (Entidade Usuário, Tabela Caronas, Tabela Perfis de Motorista) e os serviços de negócio reaproveitados (Cálculo de Rota, Operações de Carteira, Autenticação). Já as dependências simples foram reservadas para relações de uso mais específico e não reutilizado, como o acesso de um componente de negócio a uma biblioteca (Validadores de Documentos, Geolocalização) ou a uma enumeração, e para o consumo pontual das telas de apresentação sobre os componentes de negócio.

5. Reflexão e Lições Aprendidas

5.1 Principais Dificuldades

A maior dificuldade encontrada pelo grupo durante a modelagem foi lidar com o volume e a diversidade de componentes do sistema. Como o CoCar cobre um domínio amplo autenticação, gestão de usuários e motoristas, organização, trajetos, grupos de carona fixa, benefícios, financeiro e o próprio processo de match de caronas, o diagrama final acabou reunindo 35 componentes distribuídos em 10 pacotes, o que exigiu um esforço considerável de organização para que cada elemento fosse posicionado no pacote correto, de acordo com sua responsabilidade de negócio. Pensar simultaneamente numa quantidade tão expressiva de componentes tornou-se desafiador manter a consistência das relações entre eles: foi preciso decidir, para cada par de componentes, se a comunicação deveria ser representada por uma interface reutilizável ou por uma dependência simples, além de garantir que a direção de cada seta refletisse corretamente quem fornece e quem consome o serviço. Outro ponto de dificuldade foi evitar a duplicação acidental de componentes à medida que o diagrama crescia, já que, com tantos elementos espalhados pelo diagrama, era fácil recriar um componente que já existia em outra parte do diagrama. Esses desafios exigiram diversas rodadas de revisão e conferência entre os integrantes até que todas as 58 conexões planejadas entre os componentes fossem confirmadas como corretas e completas.

5.2 Desafios Conceituais

O principal desafio conceitual enfrentado pelo grupo foi definir a granularidade adequada dos componentes. Como o sistema CoCar já estava implementado fisicamente, havia a tentação inicial de mapear cada uma das 62 classes PHP do projeto diretamente para um componente do diagrama. Percebemos, no entanto, que essa abordagem geraria um número de elementos muito acima do limite estabelecido, além de resultar em um diagrama pouco legível e que não representaria adequadamente a arquitetura lógica do sistema. Foi necessário reagrupar as classes por responsabilidade de negócio, de forma que cada componente do diagrama passasse a representar um conjunto coeso de classes que juntas cumprem uma função específica.

Outro desafio conceitual relevante foi compreender quando utilizar interface e quando utilizar dependência simples. Inicialmente, o grupo tendia a usar dependência simples para todas as conexões do diagrama. Ao revisar o material da disciplina, identificamos que a interface é mais apropriada quando um mesmo serviço ou contrato é reaproveitado por mais de um componente consumidor como o acesso à tabela de usuários, utilizado por praticamente todos os componentes de autenticação enquanto a dependência simples deve ser reservada para relações mais pontuais e específicas de um único componente, como o uso de uma biblioteca ou enumeração interna. Essa

distinção também trouxe a dificuldade adicional de acertar a orientação correta dos símbolos de interface: entender que a meia-lua (interface requerida) deve encostar em quem consome o serviço, enquanto o círculo (interface fornecida) deve encostar em quem o disponibiliza, foi um ponto que exigiu atenção e correção durante a modelagem.

Por fim, o grupo também enfrentou o desafio de organizar a legibilidade de um diagrama com 35 componentes. Ficou claro que representar todas as conexões em uma única imagem tornaria o diagrama confuso, com excesso de cruzamento de linhas. A solução conceitual adotada foi utilizar a própria organização em pacotes já exigida pelo enunciado como critério de divisão visual, produzindo um diagrama macro com a relação de dependência entre os 10 pacotes (visão arquitetural geral) e diagramas complementares detalhando os componentes internos de cada pacote individualmente, evitando assim recorrer a artifícios como raias, que não se mostraram adequados à natureza das dependências do sistema.

5.3 Tomadas de Decisão

A primeira decisão estratégica do grupo foi utilizar como base o projeto CoCar já implementado em semestre anterior, em vez de modelar um sistema hipotético. Essa escolha foi justificada pelo fato de o grupo já possuir domínio profundo sobre as regras de negócio, a estrutura de dados e o funcionamento real da aplicação, o que trouxe mais segurança e coerência para a etapa de especificação e reduziu o risco de modelar componentes ou requisitos genéricos e desalinhados da prática.

A partir disso, decidimos que o critério de derivação dos componentes seria a estrutura real do código-fonte e das tabelas do banco de dados, em vez de partir de um levantamento de requisitos totalmente do zero. Essa decisão trouxe rastreabilidade ao diagrama: cada componente representado pode ser justificado por classes e tabelas que de fato existem no repositório do projeto, o que facilita a defesa do trabalho na apresentação oral, já que qualquer integrante consegue relacionar um componente do diagrama a uma parte concreta do sistema.

Também foi necessário decidir o critério de agrupamento em pacotes, optando pelo agrupamento por responsabilidade de negócio/domínio funcional (ex.: Autenticação, Trajetos, Financeiro), em vez de agrupar por camada técnica (todos os Controllers em um pacote, todos os Models em outro). Essa escolha foi justificada pelo fato de o agrupamento por domínio comunicar melhor a arquitetura do sistema para quem lê o diagrama, evidenciando quais funcionalidades de negócio existem e como elas se relacionam, o que é mais alinhado ao propósito do Diagrama de Componentes do que uma simples reorganização técnica do código.

Uma decisão importante durante a fase de desenho foi não utilizar raias para organizar visualmente os 35 componentes, mesmo essa sendo uma alternativa apresentada em sala. Ao consultar a especificação oficial da linguagem (OBJECT MANAGEMENT GROUP, 2015), o grupo identificou que o elemento *ActivityPartition* (raia) é formalmente definido na Seção 15.7.7, dentro do Capítulo 15 - Activities, não constando na Seção 11.6, que define formalmente o *Component* evidenciando que a raia não é um elemento de notação nativo do Diagrama de Componentes, mas sim do Diagrama de Atividades. O mecanismo de organização visual recomendado pela própria especificação para diagramas complexos é o *Package*, definido no Capítulo 12. Após essa análise, o grupo concluiu que as dependências entre os componentes do CoCar também não seguem um fluxo estritamente hierárquico em camadas há casos de componentes de um mesmo "nível" dependendo entre si, e até

dependências em via dupla entre pacotes (como entre Autenticação e Usuários/Motoristas). Forçar essas relações dentro de raias horizontais geraria um emaranhado de linhas cruzando as divisórias, tornando o diagrama mais confuso do que organizado. Como o enunciado já exige a divisão em pacotes mecanismo que, conforme demonstrado, é o formalmente indicado pela própria UML para esse fim, optamos por usar essa divisão como o único critério de organização visual, evitando redundância e mantendo a leitura mais limpa.

Por fim, para lidar com a legibilidade de um diagrama com 35 componentes, o grupo decidiu apresentar o diagrama em dois níveis: um diagrama macro, mostrando apenas as dependências entre os 10 pacotes, e diagramas complementares detalhando os componentes internos de cada pacote individualmente. Essa decisão foi justificada pela necessidade de equilibrar completude (mostrar todas as 58 conexões exigidas) com clareza visual, permitindo que o leitor entenda primeiro a arquitetura geral do sistema e depois aprofunde nos detalhes de cada domínio funcional específico.

6. Referências

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (UML), Version 2.5. OMG Document Number: formal/2015-03-01, mar. 2015. Disponível em: <https://www.omg.org/spec/UML/2.5/PDF>. Acesso em: 29 ago. 2026.